



**NMAM INSTITUTE
OF TECHNOLOGY**

**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING**

REPORT ON

CD Mini Project

Carried out on

**NUMBER OF DIGITS IN A BINARY NUMBER FOR A
GIVEN DECIMAL NUMBER**

COURSE CODE : CS3101-1

COURSE NAME : COMPILER DESIGN

Semester : VI SEM Section : C

Submitted To:

Dr. Raju K

Professor

Department of Computer Science and Engineering

Submitted By:

SACHIN

NNM23CS162

RISHAB RAJ K

NNM23CS154

SAMARTH VENKAPPA PATIL

NNM24CS515

Date of Submission: 21-04-2026



**NMAM INSTITUTE
OF TECHNOLOGY**

**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING**

CERTIFICATE

This is to certify that Rishab Raj K NNM23CS154, Sachin NNM23CS162 and Samarth Venkappa Patil NNM24CS515 of 6th semester B.Tech, a bonafide student of NMAM Institute of Technology, Nitte, has completed CD mini project on "Designing a Compiler for a Hypothetical Language" during March 2025 -April 2025 fulfilling the partial requirements for the award of degree of Bachelor of Technology in Computer Science & Engineering at NMAM Institute of Technology, Nitte

Signature of Course Instructor

Signature of HOD

2

DEPT.OF CSE, NMAMIT, Nitte

ACKNOWLEDGEMENT

The satisfaction that accompanies the completion of any task would be incomplete without the mention of all the people, without whom this endeavour would have been a difficult one to achieve. Their constant blessings, encouragement, guidance and suggestions have been a constant source of inspiration.

First and foremost, my gratitude to my **Dr. RAJU K** for his constant guidance throughout the course of this project, for the valuable suggestions and all the necessary help required.

I also take this opportunity to express a deep sense of gratitude to the internship coordinators for their valuable guidance and support.

My sincere thanks to our Head of the Department of Computer Science and Engineering, **Dr. Jyothi Shetty**, Principal, **Dr. Niranjana N. Chiplunkar** and Vice Principal, **Dr. Nagesh Prabhu** for permitting this project in our college and providing all necessary facilities.

Finally, thanks to *staff members* and *our friends* for their honest opinions and suggestions throughout the course of our project.

RISHAB RAJ K (NNM23CS154)

SACHIN (NNM23CS162)

SAMARTH VENKATAPATIL (NNM24CS515)

ABSTRACT

This project introduces a web-based Compiler Simulator designed to visually demonstrate the fundamental front-end phases of compiler design: Lexical Analysis and Syntax Analysis. Built with a hybrid architecture, the system utilizes C for highly efficient back-end compiler logic and HTML, CSS, and JavaScript for an interactive frontend interface. The simulator processes a simplified, C-like structural language in two core stages. First, the Lexer scans the input source code to classify distinct elements into valid tokens—such as keywords, identifiers, and operators—while dynamically constructing a symbol table. Following this, the Parser evaluates the tokenized output against predefined grammatical rules to ensure structural validity, checking for essential components like main functions, proper block scoping, and loops. By integrating these backend compilation phases with a responsive web interface, the project allows users to dynamically execute code, observe real-time tokenization and grammar validation, and export reports to PDF. Ultimately, it serves as an accessible and effective educational tool linking theoretical compiler concepts with practical, visual implementation.

TABLE OF CONTENTS

Title Page.....	1
Certificate.....	2
Acknowledgement.....	3
Abstract.....	4
Table of Contents.....	5
Introduction.....	6
Problem Statement.....	7
Objectives.....	8
Hardware/Software Requirement.....	9
Methodology.....	10
Implementation Details.....	11
Results.....	12
Conclusion.....	13
References.....	14

INTRODUCTION

A compiler is a foundational and highly complex piece of system software that acts as a bridge between human logic and machine execution. It is responsible for translating high-level, human-readable source code into low-level machine code that a computer's hardware can process. The compilation process is inherently sequential and divided into several distinct phases. The very first phases—Lexical Analysis and Syntax Analysis (often referred to as the compiler's "front-end")—are critical. They are responsible for meticulously scanning raw text, identifying meaningful components, and validating the grammatical structure before any actual translation occurs. Because these processes happen rapidly and internally, it can be extremely difficult for students and new developers to visualize how compilers interpret and validate code.

PROBLEM STATEMENT

Design a compiler(Lexical and Parser phase) for the following hypothetical languages. Number of digits in a binary number for a given decimal number

```
int main()
```

```
begin
```

```
int count=1;
```

```
while(n>1)
```

```
count=count+1;
```

```
n=n/2;
```

```
end while
```

```
return count
```

```
end
```

OBJECTIVES

The main objective of this project is to design and implement the front-end phases of a compiler, specifically the lexical analyzer and syntax analyzer, for a hypothetical programming language. This project aims to provide a clear understanding of how source code is processed in the early stages of compilation.

- The specific objectives of the project are as follows:
- To understand the basic concepts of compiler design, especially the roles of lexical and syntax analysis.
- To develop a lexical analyzer that can read a program and break it into meaningful tokens such as keywords, identifiers, operators, symbols, and constants.
- To design a syntax analyzer (parser) that checks whether the input program follows the correct grammatical structure defined by the language.
- To implement the entire system using the C programming language without relying on automated tools such as Lex and YACC, thereby gaining deeper insight into manual compiler construction.
- To modify the system to accept live user input through standard input, making the compiler interactive and closer to real-world applications.

HARDWARE/SOFTWARE REQUIREMENTS

Hardware Requirements

- Processor: Intel i3 or above
- RAM: Minimum 4 GB
- Hard Disk: At least 500 MB free space
- Input Devices: Keyboard
- Output Devices: Monitor

Software Requirements

- Operating System: Windows 10 / Windows 11 / Linux
- Programming Language: C
- Compiler: GCC (MinGW for Windows)
- IDE/Editor: Visual Studio Code

METHODOLOGY

The methodology of this project involves designing and implementing the front-end phases of a compiler, namely lexical analysis and syntax analysis, for a hypothetical programming language. Initially, the given problem and program structure were analyzed to identify various components such as keywords, identifiers, operators, and control structures. Based on this, token definitions and basic grammar rules were established. The lexical analyzer was then developed using the C programming language to read the source code through standard input and convert it into meaningful tokens. Following this, a syntax analyzer (parser) was implemented to validate the structure of the program by checking constructs like the main function, while loop, assignment statements, and return statement. The system was designed without using tools like Lex and YACC to ensure a better understanding of manual compiler construction. The program was tested with both valid and invalid inputs to verify correct token generation and error detection. This approach demonstrates the working of a simple compiler front-end in an interactive and efficient manner.

IMPLEMENTATION DETAILS

The system is developed using Python 3 with key built-in modules — `re` for token pattern matching, `enum` for token type definitions, `dataclasses` for AST node structures, and `sys` for command-line handling.

The Lexer scans source code in a single pass, producing a token stream with type, value, line, and column metadata. The Parser reads this stream and applies grammar rules recursively, building AST nodes such as `ArrayDecl`, `ForStatement`, `IfStatement`, and `Assignment`. Expression parsing handles binary operators with correct precedence

Error handling raises descriptive exceptions with exact line and column numbers at both the lexer and parser levels. The compiler driver runs both phases sequentially and prints the token table followed by the indented AST. Source input can be provided either as a built-in sample or via an external `.hyp` file through the command line.

RESULTS

```
Active code page: 65001
```

```
C:\Users\patil\OneDrive\Desktop\Cd project>lexer
```

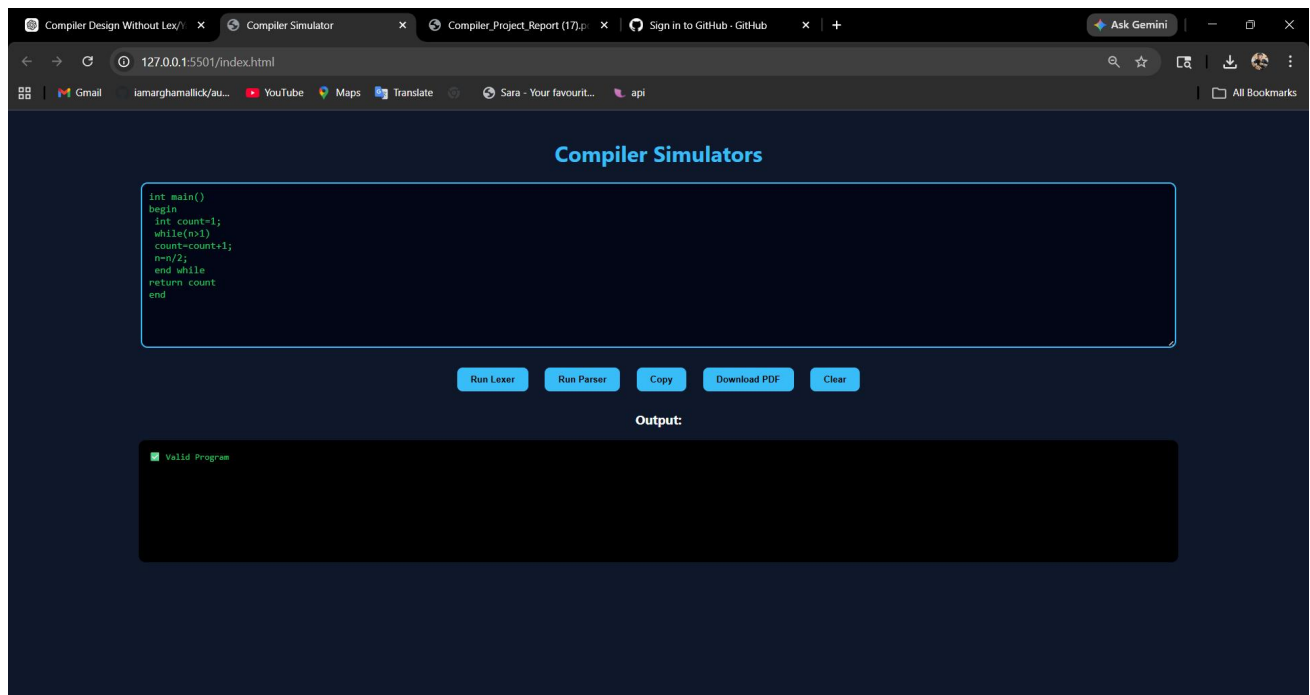
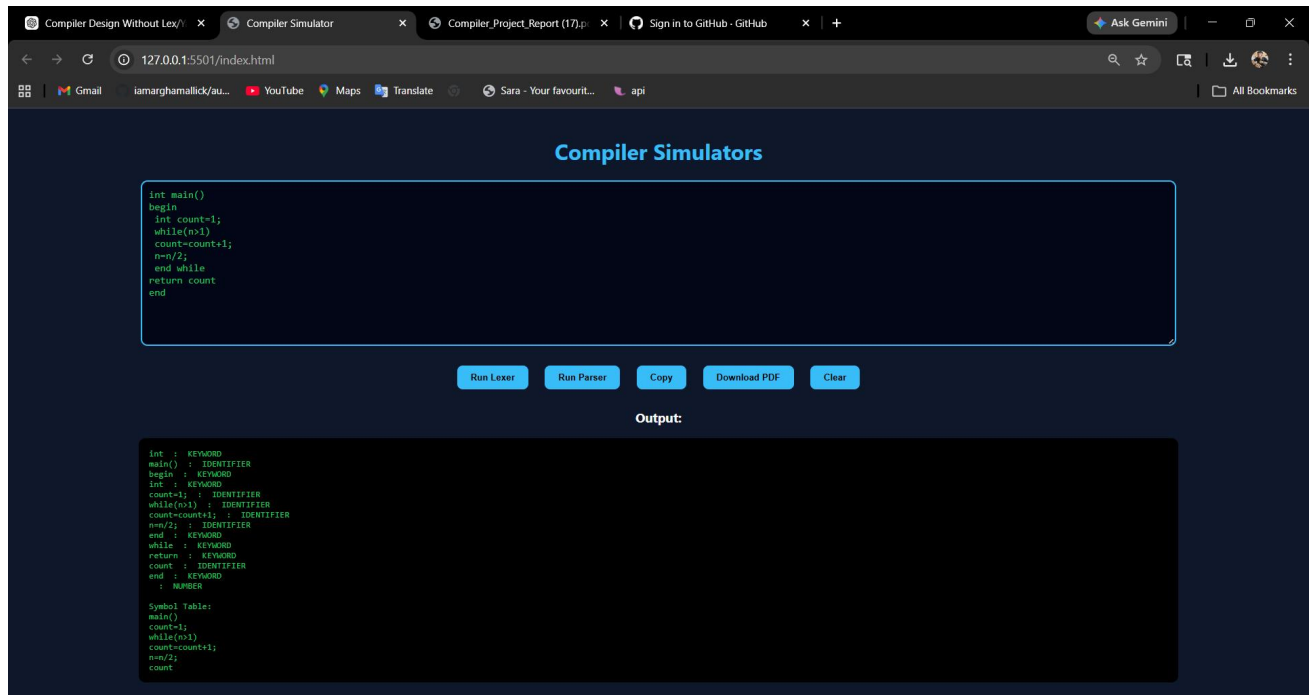
```
int → KEYWORD  
main → IDENTIFIER  
begin → KEYWORD  
int → KEYWORD  
count → IDENTIFIER  
= → OPERATOR  
1 → NUMBER  
while → KEYWORD  
n → IDENTIFIER  
> → OPERATOR  
1 → NUMBER  
count → IDENTIFIER  
= → OPERATOR  
count → IDENTIFIER  
+ → OPERATOR  
1 → NUMBER  
n → IDENTIFIER  
= → OPERATOR  
n → IDENTIFIER  
/ → OPERATOR  
2 → NUMBER
```

```
2 → NUMBER  
end → KEYWORD  
while → KEYWORD  
return → KEYWORD  
count → IDENTIFIER  
end → KEYWORD
```

```
✦ Symbol Table:
```

```
main  
count  
n
```

```
C:\Users\patil\OneDrive\Desktop\Cd project>
```





NMAM Institute of Technology, Nitte

Name: Samarth Venkappa Patil , Rishab Raj K , Sachin
Department: Computer Science Engineering

Date: 4/21/2026

Design and Implementation of a Simple Compiler

Input Program:

```
int main ()  
begin  
  int count=1;  
  while (n>1)  
    count=count+1;  
    n=n/2;  
  end while  
  return count  
end
```

Output:

```
int : KEYWORD  
main() : IDENTIFIER  
begin : KEYWORD  
int : KEYWORD  
count=1; : IDENTIFIER  
while (n>1) : IDENTIFIER  
count=count+1; : IDENTIFIER  
n=n/2; : IDENTIFIER  
end : KEYWORD  
while : KEYWORD  
return : KEYWORD  
count : IDENTIFIER  
end : KEYWORD  
 : NUMBER
```

Symbol Table:

```
main()  
count=1;  
while (n>1)  
count=count+1;  
n=n/2;  
count
```

Guide Signature

Dr. Raju K
Professor / System Administrator
Dept. of Computer Science and Engineering
NMAM Institute of Technology, Nitte.

CONCLUSION

This project successfully demonstrates the design and implementation of a simple compiler front-end for a hypothetical programming language. The lexical analyzer effectively identifies and classifies tokens such as keywords, identifiers, operators, symbols, and constants from the input program, while the syntax analyzer validates the structure of the program based on predefined grammatical rules. The use of live user input makes the system interactive and closer to real-world compiler behavior. By implementing the compiler manually without using tools like Lex and YACC, the project provides a clear understanding of the fundamental concepts involved in compiler design. Overall, this project bridges the gap between theoretical knowledge and practical application, and serves as a strong foundation for learning advanced compiler construction techniques.

REFERENCES

- <https://www.javatpoint.com/compiler-tutorial>
- https://www.tutorialspoint.com/compiler_design/index.html
- <https://www.geeksforgeeks.org/compiler-design-tutorials/>
- <https://pypi.org/project/tabulate/>